The Boundary of Synchronization Dominance: A Depth Characterization for Divisible-Work Scheduling on DAGs

Eric Bowman[1*]

[1]King, Berlin, Germany.

Corresponding author(s). E-mail(s): eric.bowman@king.com;

Abstract

Permutation schedules pointwise-dominate in the divisible-work two-stage assembly flow shop. We determine exactly how far this extends when the two-stage structure generalizes to chains and to arbitrary DAGs of stages under a preemptive divisible-work model. Our main result is a complete characterization: fully synchronized schedules—one common priority order at every stage—are makespan-optimal at every sink for every assignment of work if and only if the DAG contains no directed path through three stages. The positive direction reduces, sink by sink, to a fluid form of the two-stage capacity argument; the negative direction embeds a three-stage chain counterexample along any longer path via a scaling argument, with relative optimality gap approaching **1/8** of the synchronized makespan. The characterization corrects a conjecture from our earlier preprint: makespan dominance was conjectured for diamond and fork-join–fork-join topologies on computational evidence whose sweeps held source works fixed; with source works adversarial, failures are in fact common. For chains we show makespan dominance holds at length two and fails from length three; for total flowtime we prove dominance for two jobs with concordant work profiles and show it fails beyond. Finally we quantify the practical stakes: a composite policy that synchronizes only the source stages and serves downstream work in arrival order provably reproduces the synchronized schedule on single-source DAGs, is within **0.06%** of the enumerated optimum on average over an exhaustive instance family, retains most of its value under $\pm$**50%** work-estimation error, and the residual gap concentrates on rare, identifiable cross-stage work inversions.

Keywords: assembly flow shop, permutation schedule, divisible work, DAG scheduling, synchronization, makespan

MSC Classification: 90B35

1 Introduction

For the two-stage assembly flow shop with divisible work—m parallel machines of k identical workers feeding a single assembly stage—permutation schedules dominate pointwise: for every schedule there is a schedule in which all stage-1 machines share one job order and every job's assembly completion is no later (Bowman, 2026). The result is constructive, holds for every regular objective, and is formalized in Lean 4. It invites an obvious question: how much of the organization-like structure around the two-stage core can the dominance survive? Deeper pipelines? Intermediate parallel layers? Branching without reconvergence?

This paper answers the question completely for the makespan criterion in the preemptive divisible-work model. The answer is a sharp structural boundary:

> *Full synchronization—one common priority order at every stage—is makespan-safe for every work assignment if and only if no directed path of the stage DAG passes through three stages.*

The "if" direction is a fluid form of the two-stage capacity argument applied sink by sink (Theorem 10). The "only if" direction is new: any three-stage path admits an embedding of a chain counterexample, made robust to the surrounding DAG by a scaling argument, and the optimality gap approaches 1/8 of the synchronized makespan (Theorem 9). The mechanism is *overtaking*: when a job is light at one stage and heavy at the next, optimal schedules let it pass its neighbour between stages; a common order forbids this at every depth.

The characterization corrects the record of our earlier preprint (Bowman, 2026), which conjectured—from more than 2×10^7 verified instances—that makespan dominance survives on diamond and fork-join–fork-join topologies, attributing the apparent rescue to intermediate parallel width. The conjecture is false, and the evidence was an artifact of sweep design: the exhaustive sweeps varied work only at non-source stages. Embedding the chain counterexample with adversarial *source* works produces failures in all of these topologies, at rates near 5% of random instances; an exhaustive small-instance sweep including source works finds failures at 0.55% density (Section 6). Intermediate width neither helps nor hurts: depth alone decides.

Around the makespan boundary we assemble the adjacent dominance landscape. For chains of stages, makespan dominance holds at length two and fails from length three (Section 4). For total flowtime on chains, synchronization is optimal for two jobs whose work profiles are *concordant*—the lighter job is lighter at every stage—and fails beyond, both without concordance and with three concordant jobs (Section 7).

Finally, because the negative results say only that full synchronization is not *free* at depth three, we quantify what it actually costs and what a manager should do instead (Section 8). A composite policy—one shared priority order at the source stages, first-in-first-out by arrival everywhere downstream—provably reproduces the fully synchronized schedule on single-source DAGs (Lemma 23); over an exhaustive family of 65,536 instances it is within 0.06% of the enumerated optimum on average and optimal outright in 99.4% of instances, with the same pattern across chain and two-source fork-join families; and its near-optimality survives ±50% work-estimation error. Where full synchronization does fail, the gap is material (mean 5.5–10.4% across families) but rare (0.55–10% of instances) and identifiable: it requires a cross-stage work inversion, which deliberate expediting can capture.

2 Related work

The two-stage assembly flow shop was introduced by Lee et al. (1993). Potts et al. (1995) proved strong NP-hardness for $m \geq 2$ and the foundational dominance result: permutation schedules suffice for makespan, reducing the search space from $(n!)^m$ to $n!$. Tozkapan et al. (2003) extended permutation dominance to total weighted flowtime; Hariri and Potts (1997) and Hadda et al. (2024) developed branch-and-bound methods built on dominance rules. Koulamas and Kyparisis (2007) studied uniform parallel machines at stage 1. Our companion paper (Bowman, 2026) extends the dominance pointwise to a divisible-work model with k identical workers per machine and machine-dependent works; the present paper takes that result as its starting point and maps its outer boundary.

Adjacent literatures do not address the question. The malleable- and divisible-load literature (Drozdowski, 2009) studies divisible work on parallel processors without the assembly or DAG-precedence structure between stages. Distributed assembly permutation flow shops (Hatami et al., 2013) concern multiple factories rather than per-stage priority synchronization. Zhang and Chen (2022) schedule assembly with parallel worker teams in a multi-objective setting without dominance results. The survey of Komaki et al. (2019) covers assembly flow shop variants; neither the divisible-work model nor the question of when a single shared priority order is lossless across a DAG of stages appears there. Al-Anzi and Allahverdi (2007) use permutation dominance to restrict heuristic search in the two-stage setting with setups. In the three-field notation of Graham et al. (1979) our positive results live in assembly-type extensions of $F2 \mid \text{div} \mid C_{\max}$; the negative results concern arbitrary stage DAGs under a preemptive fluid relaxation.

3 The fluid model and basic lemmas

Definition 1 (Fluid preemptive priority DAG model) A DAG $G = (V, E)$ of *stages*; stage v has $k_v \geq 1$ identical workers and work $w_{v,j} > 0$ for each job $j \in [n]$. Job j *arrives* at v at $a_v(j) = \max_{u:(u,v)\in E} C_u(j)$ (at sources, $a_v(j) = 0$). A schedule assigns each stage a priority order $\beta_v \in \mathfrak{S}_n$; at every instant, stage v devotes all k_v workers (total rate k_v) to the highest-β_v-priority job that has arrived and is incomplete at v. $C_v(j)$ denotes the completion time of j at v. A schedule is *synchronized* (a permutation schedule) if $\beta_v = \pi$ for all v. For a sink t, the makespan at t is $\max_j C_t(j)$. We say *makespan dominance holds at sink* t for an instance if

$$\min_{\pi \in \mathfrak{S}_n} \max_j C_t^{\pi}(j) \;=\; \min_{\boldsymbol{\beta}} \max_j C_t^{\boldsymbol{\beta}}(j).$$

Each stage's trajectory is piecewise linear with breakpoints only at arrivals and completions, so all completion times are well defined and finite. Two elementary properties of the dynamics: they are *time-invariant* (shifting all arrivals at a stage by ε shifts its completions by exactly ε) and *positively homogeneous* (scaling all works by

M scales all completion times by M). Since a stage's dynamics depend on works only through the normalized quantities $w_{v,j}/k_v$, we refer to these as *processing times* and state instances either way.

The workhorse is a reduction of each priority class to a single-server fluid queue.

Lemma 2 (Prefix-queue reduction) *Fix a stage with priority order β, rate k, works $w_j > 0$, and arrival times $a(j)$. For $p \in [n]$ let $P_p = \{\beta(1), \ldots, \beta(p)\}$, $W_p = \sum_{j \in P_p} w_j$, and let $A_p(t) = \sum_{j \in P_p,\, a(j) \le t} w_j$ be the arrived prefix work. Let $S_p(t)$ denote the cumulative service delivered to jobs of P_p by time t. Then:*

(i) *S_p is continuous, $S_p(0) = 0$, $S_p(t) \le A_p(t)$ for all t, and between breakpoints*

$$\tfrac{d}{dt} S_p(t) \;=\; k \cdot \mathbf{1}\big[A_p(t) > S_p(t)\big].$$

(ii) *The time at which all of P_p is complete satisfies*

$$M_p \;:=\; \max_{j \in P_p} C(j) \;=\; \inf\{\, t : S_p(t) = W_p \,\}.$$

(iii) (Invariance.) *The stage makespan M_n depends only on the arrival times and works, not on β.*

(iv) (Monotonicity and Lipschitz bound.) *Each M_p is non-decreasing in every arrival time, and $|M_p(a') - M_p(a)| \le \sup_j |a'(j) - a(j)|$.*

(v) (Ordered arrivals.) *If $a(\beta(1)) \le \cdots \le a(\beta(n))$, then no job is ever preempted, $C(\beta(p)) = M_p$ for every p, and the completions obey the max-plus recurrence*

$$C(\beta(p)) = \max\big(a(\beta(p)),\, C(\beta(p-1))\big) + w_{\beta(p)}/k, \qquad C(\beta(0)) := 0.$$

Proof (i) Whenever some job of P_p has arrived and is incomplete, the highest-priority arrived incomplete job lies in P_p, so the stage serves a job of P_p at full rate k; otherwise it serves outside P_p or idles, and S_p is flat. The condition "some job of P_p arrived and incomplete" is exactly $A_p(t) > S_p(t)$, and service cannot exceed arrived work, giving $S_p \le A_p$.

(ii) All of P_p is complete at time t if and only if all arrived prefix work has been served and no prefix work remains to arrive, i.e. $S_p(t) = W_p$; the infimum is attained by continuity of S_p.

(iii) For $p = n$, A_n is the total arrived work, which does not depend on β, and by (i) the trajectory S_n is determined by A_n; by (ii), M_n is determined by S_n.

(iv) Let $a \le a'$ componentwise, with arrived-work curves $A_p \ge A'_p$ pointwise, and trajectories S_p, S'_p from (i). Set $\varphi(t) = \max\big(S'_p(t) - S_p(t), 0\big)$, a continuous piecewise-linear function with $\varphi(0) = 0$. At any non-breakpoint t with $S'_p(t) > S_p(t)$ we have $A_p(t) \ge A'_p(t) \ge S'_p(t) > S_p(t)$, so $\frac{d}{dt} S_p(t) = k \ge \frac{d}{dt} S'_p(t)$ and $\varphi'(t) \le 0$; where $S'_p \le S_p$, φ is locally zero. Hence $\varphi \equiv 0$, i.e. $S'_p \le S_p$ pointwise, and by (ii) $M_p(a) \le M_p(a')$. The Lipschitz bound follows from monotonicity and time-invariance: if $a \le a' \le a + \varepsilon \mathbf{1}$ then $M_p(a) \le M_p(a') \le M_p(a) + \varepsilon$, and the general case reduces to this with $\varepsilon = \sup_j |a'(j) - a(j)|$ after replacing a by the componentwise minimum.

(v) With β-ordered arrivals, whenever $\beta(p)$ is present, every incomplete higher-priority job has already arrived, so $\beta(p)$ is served only when all of P_{p-1} is complete, and from that moment it is served at rate k without interruption. Hence $\beta(p)$ starts at $\max\big(a(\beta(p)), C(\beta(p-1))\big)$ (using $C(\beta(p-1)) = M_{p-1}$ inductively), runs for $w_{\beta(p)}/k$, and completes last among P_p, giving $C(\beta(p)) = M_p$ and the recurrence. □

Part (iv) is deliberately stated for the prefix maxima M_p and not for individual completion times: individual completions are *not* monotone in arrivals (delaying a high-priority job's arrival can let a low-priority job escape preemption and finish earlier). Part (v) recovers per-job control exactly when arrivals respect the priority order, which is the regime our positive arguments arrange.

Lemma 3 (Light stages perturb little) *Let stage v have total work $\overline{W}_v = \sum_j w_{v,j}$. Then for every priority order and every job j:* (i) $C_v(j) \le a_v(j) + \overline{W}_v/k_v$, *and* (ii) $\max_j C_v(j) \le \max_i a_v(i) + \overline{W}_v/k_v$.

Proof (i) From $a_v(j)$ until $C_v(j)$, job j is present and incomplete, so the stage continuously serves *some* job at rate k_v; total work is $\overline{W}_v$, so the busy time is at most $\overline{W}_v/k_v$. (ii) After $\max_i a_v(i)$ all work has arrived; work conservation exhausts it within $\overline{W}_v/k_v$. □

4 Chains: dominance to depth two, failure from depth three

A *chain* of length L is the path DAG $1 \to 2 \to \cdots \to L$ in Definition 1: stage ℓ has $k_\ell \ge 1$ workers and works $w_{\ell,j}$, all jobs are released to stage 1 at time 0, and stage $\ell \ge 2$ can begin job j only after stage $\ell - 1$ completes it. This is the two-stage assembly model with $m = 1$ when $L = 2$, and the simplest setting in which depth can be isolated from width.

Lemma 4 (Priority-order completion) *Under a synchronized schedule with order π on a chain, jobs complete each stage in priority order: $C_\ell(\pi(1)) \le \cdots \le C_\ell(\pi(n))$ for every stage ℓ.*

Proof By induction on ℓ. At stage 1 all jobs arrive at time 0, which is trivially π-ordered; at stage $\ell \ge 2$ the arrivals are the stage-$(\ell-1)$ completions, π-ordered by the inductive hypothesis. In both cases Lemma 2(v) applies and yields π-ordered completions. □

Theorem 5 (Two-stage chain makespan dominance) *For $L = 2$, the optimal makespan over all per-stage orderings equals the optimal makespan over synchronized schedules.*

Proof Consider any schedule with orderings β_1, β_2. By Lemma 2(iii), the makespan at stage 2 depends only on the arrival times at stage 2 (the stage-1 completions) and the stage-2 works, not on β_2. Therefore the synchronized schedule (β_1, β_1) achieves the same makespan as (β_1, β_2). Since this holds for every β_1, the minimum over synchronized schedules equals the minimum over all schedules. □

For longer chains the natural inductive extension—apply the $(L-1)$-stage result, then stage-L invariance—requires that an ordering good for stages $1,\dots,L-1$ is also good for stage L. This fails when work profiles invert across stages, and the failure is not an artifact of exotic objectives: it already defeats the makespan.

Proposition 6 (Failure of chain dominance for $L \geq 3$) *For $L \geq 3$, there exist instances where no synchronized schedule achieves the optimal makespan, and instances where no synchronized schedule achieves the optimal total weighted completion time.*

Proof Counterexample: $L = 3$, $n = 2$, $k_\ell = 2$, $w = [(1,2),\ (4,1),\ (1,2)]$, i.e. processing times $(0.5, 1.0)$, $(2.0, 0.5)$, $(0.5, 1.0)$ for jobs $(1, 2)$ stage by stage.

Under the synchronized order $(1, 2)$: at stage 1, job 1 completes at 0.5 and job 2 at 1.5. At stage 2, job 1 occupies $[0.5, 2.5]$; job 2 then occupies $[2.5, 3.0]$. At stage 3, job 1 occupies $[2.5, 3.0]$ and job 2 $[3.0, 4.0]$. Completions $(3.0, 4.0)$: makespan 4.0, flowtime 7.0.

Under the synchronized order $(2, 1)$: stage 1 gives $(1.5, 1.0)$; stage 2: job 2 occupies $[1.0, 1.5]$, job 1 $[1.5, 3.5]$; stage 3: job 2 $[1.5, 2.5]$, job 1 $[3.5, 4.0]$. Completions $(4.0, 2.5)$: makespan 4.0, flowtime 6.5.

Under the unsynchronized orders $(1, 2) \mid (2, 1) \mid$ any (the stage-3 ordering is irrelevant for the makespan by Lemma 2(iii)): stage 1 gives $(0.5, 1.5)$. At stage 2, job 1 starts at 0.5; job 2 arrives at 1.5 with higher priority and preempts it (job 1 has 1.0 of 2.0 processing done); job 2 completes at 2.0; job 1 resumes and completes at 3.0. At stage 3 (order $(2, 1)$), job 2 arrives at 2.0 and completes at 3.0; job 1 arrives at 3.0 and completes at 3.5. Completions $(3.5, 3.0)$: makespan 3.5, flowtime 6.5.

Hence the unsynchronized schedule achieves strictly better makespan $(3.5 < 4.0)$ and equal-or-better flowtime; enumeration over all $2^3 = 8$ per-stage orderings confirms no schedule synchronized across the three stages achieves makespan below 4.0. For weighted completion time, the stage-3 completion vectors are $C^{(1,2)} = (3.0, 4.0)$, $C^{(2,1)} = (4.0, 2.5)$, and $C^{\text{unsync}} = (3.5, 3.0)$; with weights $v = (3, 2)$ the synchronized schedules both give 17.0 while the unsynchronized schedule gives 16.5.

For $L > 3$, append stages at which both jobs have work $\varepsilon > 0$. Arrivals at each appended stage are the previous stage's completions and completions only increase along the chain, so under every synchronized order the final makespan is still at least 4.0 and the weighted completion time at least 17.0; extending the unsynchronized schedule through the appended stages adds at most $2\varepsilon/k_\ell$ per stage to each completion (Lemma 3(i)). For ε small, both strict separations persist. $\square$

Remark 7 (Mechanism: cross-stage work inversion) The counterexample exploits preemption at an intermediate stage. Job 1 is light at stage 1 but heavy at stage 2; job 2 is the reverse. The unsynchronized schedule uses opposite priorities at stages 1 and 2: stage 1 completes the light job 1 early so it flows downstream; at stage 2, job 2 overtakes it, completing its short stage-2 work immediately and feeding stage 3 sooner. A synchronized schedule cannot express this overtaking. Systematic search over 46,656 chain configurations ($L = 3$, $n = 2$, $k = 2$, work values 1–6) found 1,414 makespan counterexamples and 1,016 flowtime counterexamples; the largest makespan gap in that range is 1.0 (e.g. synchronized 5.5 vs. unsynchronized 4.5). By positive homogeneity the $k = 3$ sweep yields the same counts, with gaps scaled by 2/3.

Remark 8 (Pointwise domination fails already at $L = 2$) Theorem 5 is about the makespan, and that is not an accident of presentation. In the fully synchronized sense of Definition 1—the sink must use the common order too—*pointwise* domination fails already on two-stage chains. Take $L = 2$, $n = 2$, $k_\ell = 1$, $w_1 = (1, 2)$, $w_2 = (3, 1)$, and the schedule $\beta_1 = (1, 2)$, $\beta_2 = (2, 1)$: stage-1 completions are $(1, 3)$; stage 2 serves job 1 on $[1, 3)$, is preempted by job 2 on $[3, 4]$, and finishes job 1 at 5, giving completions $(5, 4)$. The synchronized order $(1, 2)$ gives $(4, 5)$ (worse for job 2); the order $(2, 1)$ gives $(6, 3)$ (worse for job 1). Neither dominates pointwise. This does not contradict the pointwise theorem of Bowman (2026): there the constructed permutation schedule synchronizes the *upstream* machines and *inherits* the downstream sequence of the original schedule, whereas full synchronization forces the downstream order as well. Makespan dominance is the strongest form that survives full synchronization, and it is the form our characterization settles.

5 The boundary: a depth characterization

We now place the chain counterexample inside an arbitrary DAG. Throughout this section, fix a directed path $u \to v \to x$ in G (three distinct stages), let $n \geq 2$, and call jobs 1 and 2 the *heavy* jobs and jobs $3, \dots, n$ *padding* jobs. Given a scale $M \in \mathbb{Z}_{>0}$, define the instance I_M:

$$w_u = (k_u M,\, 2k_u M), \quad w_v = (4k_v M,\, k_v M), \quad w_x = (k_x M,\, 2k_x M)$$

for the heavy jobs (so the path processing times are $(M, 2M)$, $(4M, M)$, $(M, 2M)$: the instance of Proposition 6 scaled by $2M$), work 1 for every padding job at u, v, x, and work 1 for every job at every off-path stage. Let

$$\delta \;:=\; n\,|V| \;\geq\; \sum_{y \notin \{u,v,x\}} \overline{W}_y / k_y \;+\; \max_{s \in \{u,v,x\}} \overline{W}_s^{\text{pad}} / k_s,$$

a crude aggregate bound on all off-path and padding work (normalized); here $\overline{W}_s^{\text{pad}} = n - 2$ is the padding work at a path stage. Two consequences of Lemma 3 used repeatedly: (a) every off-path stage all of whose ancestors are off-path completes all jobs by δ (sum Lemma 3(ii) along ancestors); in particular all arrivals at u are at most δ; (b) for any schedule and any job j, a route through off-path stages delays j by at most δ in aggregate beyond its completion at the last path stage on the route (sum Lemma 3(i)).

Theorem 9 (Any three-stage path defeats full synchronization) *Let G contain a directed path $u \to v \to x$ and let the worker counts $k_\cdot \geq 1$ be arbitrary. For every $n \geq 2$ and every integer $M > 6\delta$, the instance I_M satisfies: at every sink t reachable from x,*

$$\min_{\pi} \max_{j} C_t^{\pi}(j) \;\geq\; 8M - \delta, \qquad \min_{\boldsymbol{\beta}} \max_{j} C_t^{\boldsymbol{\beta}}(j) \;\leq\; 7M + 5\delta.$$

In particular makespan dominance fails at t, every synchronized schedule exceeds the optimum by at least $M - 6\delta$, and the relative gap $1 - C_t^{\text{opt}} / C_t^{\text{sync}}$ tends to $1/8$ as $M \to \infty$.

Proof The heavy jobs' processing times at the path stages are $(M, 2M)$, $(4M, M)$, $(M, 2M)$: the instance of Proposition 6 scaled by $2M$, whose isolated synchronized and optimal makespans are $8M$ and $7M$. We use two facts valid for every priority schedule: for all v, j,

(F1) $C_v(j) \geq a_v(j) + w_{v,j}/k_v$, and

(F2) if j' has lower β_v-priority than j, then j' receives no service at v during any interval in which j is present and incomplete.

Synchronized lower bound. Fix any π and suppose first that π ranks job 1 above job 2. At u: by (F1), $C_u(1) \geq M$. By (F2), job 2 is served at u only before $a_u(1) \leq \delta$ or after $C_u(1)$, so $C_u(2) \geq C_u(1) + (2M - \delta)$. At v: arrivals satisfy $a_v(1) \geq C_u(1)$ and, by consequence (b) above applied to job 1's off-path predecessors of v, $a_v(1) \leq C_u(1) + \delta$; also $a_v(2) \geq C_u(2) \geq C_u(1)+2M-\delta \geq a_v(1)+2M-2\delta > a_v(1)$. By (F1), $C_v(1) \geq a_v(1)+4M \geq C_u(1)+4M \geq 5M$. Since job 1 is present at v from $a_v(1) \leq a_v(2)$ and incomplete until $C_v(1)$, (F2) gives job 2 no service at v before $C_v(1)$, so $C_v(2) \geq C_v(1)+M \geq 6M$. At x: by (F1), $C_x(2) \geq a_x(2)+2M \geq C_v(2) + 2M \geq 8M$.

If instead π ranks job 2 above job 1, symmetrically: $C_u(2) \geq 2M$; job 1 is served at u only before $a_u(2) \leq \delta$ or after $C_u(2)$, so $C_u(1) \geq C_u(2) + M - \delta$. At v: $a_v(2) \leq C_u(2) + \delta$ and $a_v(1) \geq C_u(1) \geq a_v(2)+M-2\delta > a_v(2)$; by (F1), $C_v(2) \geq a_v(2)+M \geq C_u(2)+M$; by (F2), job 1 is served at v only after $C_v(2)$ (it arrives after $a_v(2)$, and job 2 is present and incomplete until $C_v(2)$), so $C_v(1) \geq \max(a_v(1), C_v(2)) + 4M \geq C_u(2) + M - \delta + 4M \geq 7M - \delta$. At x: $C_x(1) \geq a_x(1) + M \geq C_v(1) + M \geq 8M - \delta$.

In both cases some job completes at x no earlier than $8M - \delta$. For any sink t reachable from x, arrivals propagate: along any directed route from x to t, each stage's makespan is at least its largest arrival, hence at least $\max_j C_x(j) \geq 8M - \delta$. Note that padding jobs appear in these arguments only through the bounds $a_u(\cdot) \leq \delta$ and consequence (b); their positions in π are immaterial, since (F1) and (F2) were applied only to the heavy pair.

Constructed upper bound. Use the orders $(1, 2, \text{padding})$ at u, $(2, 1, \text{padding})$ at v and x (padding jobs in a fixed arbitrary order below both heavy jobs at all three stages), and any fixed order at off-path stages. At u: job 1 is served on arrival, so $C_u(1) = a_u(1)+M \leq M+\delta$; the heavy pair is served continuously from $\max(a_u(1), a_u(2)) \leq \delta$ until both complete, so $C_u(2) \leq 3M+\delta$. By consequence (b), heavy arrivals at v satisfy $a_v(j) \leq C_u(j)+\delta$. At v (order $(2, 1)$): $C_v(2) \leq a_v(2)+M \leq 4M+2\delta$; job 1 is served at v from $a_v(1)$ onward except while job 2 is present and incomplete, an interruption totalling at most M, so $C_v(1) \leq a_v(1)+4M+M \leq 6M + 2\delta$. At x (order $(2, 1)$): $a_x(2) \leq C_v(2) + \delta$, so $C_x(2) \leq a_x(2) + 2M \leq 6M + 3\delta$; and $a_x(1) \leq C_v(1) + \delta \leq 6M + 3\delta$, after which job 1 waits at most for job 2 and then runs, so $C_x(1) \leq \max(a_x(1), C_x(2))+M \leq 7M+3\delta$. Padding jobs: at each path stage they are served once the heavy jobs are out of the way, so all work at u, v, x is exhausted by $C_u(2) + \delta$, $C_v(1) + \delta$, $C_x(1) + \delta$ respectively (Lemma 3(ii) started from the later of the last arrival and the heavy completion); hence the makespan at x over all jobs is at most $7M+4\delta$. Downstream off-path stages to any sink add at most δ in aggregate (consequence (b)), so the makespan at t is at most $7M + 5\delta$.

Optimum lower bound. The synchronized lower bound used the priority between the heavy jobs only at u and at v, so it applies verbatim to any schedule whose orders at u and v agree on the heavy pair, giving makespan at least $8M-\delta$ at x. It remains to bound the two mixed cases. If u ranks job 2 first and v ranks job 1 first, then as before $C_u(1) \geq C_u(2)+M-\delta \geq 3M-\delta$, so $C_v(1) \geq a_v(1) + 4M \geq 7M - \delta$ and $C_x(1) \geq C_v(1) + M \geq 8M - \delta$. If u ranks job 1 first and v ranks job 2 first, then $a_v(2) \geq C_u(2) \geq C_u(1) + 2M - \delta \geq a_v(1) + 2M - 2\delta > a_v(1)$. Job 2 is present and incomplete at v throughout $[a_v(2), C_v(2)]$, an interval of length at least M, during which (F2) denies job 1 all service at v. If $C_v(1) \leq a_v(2)$ then $C_v(2) \geq$

$a_v(2) + M \geq C_v(1) + M \geq a_v(1) + 5M \geq 6M$ and $C_x(2) \geq C_v(2) + 2M \geq 8M$; otherwise job 1 is incomplete throughout that interval, which therefore lies inside $[a_v(1), C_v(1)]$, so $C_v(1) \geq a_v(1) + 4M + M \geq 6M$ and $C_x(1) \geq C_v(1) + M \geq 7M$. In every case the makespan at x—hence at every sink t reachable from x—is at least $7M - \delta$.

Conclusion. For $M > 6\delta$ the two displayed bounds are incompatible with dominance: $8M - \delta > 7M + 5\delta$. For the limit, a computation identical to the constructed upper bound but with the synchronized order $(1, 2, \text{padding})$ everywhere shows the best synchronized makespan at t is also at most $8M + 5\delta$, so

$$1 - \frac{7M + 5\delta}{8M - \delta} \leq 1 - \frac{C_t^{\text{opt}}}{C_t^{\text{sync}}} \leq 1 - \frac{7M - \delta}{8M + 5\delta},$$

and both ends tend to $1/8$ as $M \to \infty$. □

Theorem 10 (Depth two is safe) *If every directed path of G visits at most two stages (equivalently, every stage is a source or a sink), then for every work assignment, every worker assignment, and every sink t, makespan dominance holds at t.*

Proof Fix a sink t and any schedule $\boldsymbol{\beta}$. Stages that do not feed t cannot influence completions at t (stages share no workers), so restrict attention to t and its predecessors, all of which are sources (a predecessor with a predecessor would create a three-stage path): a two-stage assembly instance with all jobs available at the sources at time zero.

Let $r_j = \max_{i \in \text{pred}(t)} C_i(j)$ be the release times of $\boldsymbol{\beta}$, and let π sort jobs by non-decreasing r_j. *Capacity bound:* in $\boldsymbol{\beta}$, source i completes each of $\pi(1), \ldots, \pi(s)$ by $r_{\pi(s)}$, and its service rate is at most k_i, so $W_i(s) := \sum_{q \leq s} w_{i,\pi(q)} \leq k_i\, r_{\pi(s)}$. Under the synchronized order π, source i serves the prefix continuously from time zero (Lemma 2(v) with all arrivals zero), so $C_i^\pi(\pi(s)) = W_i(s)/k_i \leq r_{\pi(s)}$, whence the synchronized release times satisfy $r_j^\pi \leq r_j$ for every j. *Sink:* by Lemma 2(iii) the makespan at t is a function of its arrival vector only—in particular the original schedule's makespan at t is $M_t(r)$ regardless of β_t—and by Lemma 2(iv) this function is componentwise non-decreasing. Hence the synchronized makespan is $M_t(r^\pi) \leq M_t(r)$. Since $\boldsymbol{\beta}$ was arbitrary, the minimum over synchronized schedules equals the minimum over all schedules. □

Corollary 11 (Characterization) *In the model of Definition 1, the following are equivalent for a DAG G:*

(i) *for every assignment of works and workers, makespan dominance holds at every sink;*
(ii) *G contains no directed path through three stages.*

Proof (ii) ⇒ (i) is Theorem 10. If (ii) fails, G contains a path $u \to v \to x$, and Theorem 9 produces an instance and a sink at which dominance fails, contradicting (i). □

Remark 12 (Reading for organizations) Condition (ii) is precisely the "parallel build → integrate" shape. The moment any stage's output feeds a third stage of substantive work, mandating one shared order at every stage can cost up to 12.5% of the synchronized makespan (equivalently, the synchronized completion can run about 14% over the optimum). Whether deeper counterexample chains push the constant beyond 1/8 is open (Section 9).

6 Computational falsification of the diamond/FJFJ conjecture

The extended preprint of Bowman (2026) conjectured makespan dominance for diamond ($S \to \{A, B\} \to T$) and fork-join–fork-join ($\{A, B\} \to C \to \{D, E\} \to F$) topologies, supported by more than 2×10^7 verified instances. The conjecture is false; Corollary 11 replaces it (both topologies contain three-stage paths). The earlier evidence had two blind spots: the exhaustive $n = 2$ sweeps varied works only at non-source stages (source works held fixed), and the random $n = 3$ sweep diluted over a 12-dimensional work cube. Embedding the chain counterexample of Proposition 6 with adversarial *source* works produces failures immediately (Table 1; all rows verified by exhaustive enumeration of all per-stage priority orders, with $k_v = 2$ throughout).

Table 1 Embedded-chain counterexamples in topologies previously conjectured safe. Works are listed per stage as (job 1, job 2[, job 3]).

Topology	Works (stage: per job)	Sync. best	Optimum
Diamond, n=2	$A(1,2) \mid B(4,1) \mid C(1,1) \mid D(1,2)$	4.0	3.5
Diamond, n=3	as above; job 3 has work 1 throughout	4.5	4.0
Wide diamond	chain on one branch; two weak branches	4.0	3.5
FJFJ	chain works on $A \to C \to D$; B, E, F weak	4.5	4.0

Beyond the constructed instances, the failure density is substantial once source works vary. An exhaustive $n = 2$ diamond sweep over *all* works in $\{1, \ldots, 4\}^8$ (including sources; 65,536 instances) finds 362 makespan-dominance failures (0.55%). At $n = 3$ with works sampled uniformly from $\{1, \ldots, 8\}$, random sampling finds failures at a ∼5% rate (2,497 of 50,000). Moreover, the preprint's $n \geq 3$ diamond claim is not reproducible: rerunning the archived sweep generator with its original seed produces dominance failures within 200,000 instances. We therefore withdraw that empirical claim along with the conjecture it supported; a correction notice accompanies the archived preprint. The episode is itself instructive: computational evidence gathered over a high-dimensional parameter space can concentrate, unnoticed, on a lower-dimensional slice of it.

Conversely, the positive side of the characterization is corroborated beyond single sinks: an exhaustive sweep of a depth-2 two-sink topology ($\{A, B\}$ sources; $T_1 \leftarrow A$; $T_2 \leftarrow A, B$; all works in $\{1, \ldots, 4\}^8$) finds *zero* makespan-dominance failures in 65,536 instances, as Theorem 10 requires—while *pointwise* domination fails in 35.7% of them (sink by sink: some schedule and sink admit no synchronized schedule dominating every job's completion at that sink), as Remark 8 leads one to expect.

7 The flowtime boundary: concordant dominance for two jobs

The makespan boundary of Corollary 11 leaves open what synchronization costs for *flowtime* objectives on deep chains. Proposition 6 shows weighted flowtime dominance fails at $L = 3$ in general. This section proves the positive complement: for two jobs whose work profiles are *concordant*, the synchronized lighter-job-first schedule minimizes total flowtime on chains of every length, and this is sharp—it fails for three concordant jobs and fails without concordance.

Throughout, an L-stage chain has stages $1, \dots, L$ with $k_\ell \geq 1$ workers; two jobs A and B have works $w_\ell(A), w_\ell(B)$ at stage ℓ, with processing times $p_\ell = w_\ell(A)/k_\ell$ and $q_\ell = w_\ell(B)/k_\ell$. Profiles are *concordant* if $p_\ell \leq q_\ell$ for every ℓ (A is always the lighter job). A *per-stage schedule* is a vector $\sigma \in \{A, B\}^L$ giving the priority job at each stage; the *synchronized* schedule is $\sigma^* = (A, \dots, A)$. Jobs are released to stage 1 at time 0, and $C^\sigma_\ell(\cdot)$ denotes stage-ℓ completion times.

Theorem 13 (Concordant flowtime dominance, $n = 2$) *For any L-stage concordant chain with two jobs, the synchronized schedule σ^* minimizes total flowtime: $C^{\sigma^*}_L(A) + C^{\sigma^*}_L(B) \leq C^\sigma_L(A) + C^\sigma_L(B)$ for every per-stage schedule σ.*

The proof occupies the rest of this section. We may assume $p_\ell > 0$; if $p_\ell = 0$ at some stage, job A passes through it instantly under every schedule and the stage can be deleted.

7.1 Single-stage transitions

At a single stage with processing times $p \leq q$ and arrival times a_A, a_B, only the first-arriving job is available between the two arrivals and is processed regardless of priority. The completion times in the cases we need are as follows; in each overlap case the server is busy continuously from the first arrival to the last completion.

Lemma 14 (Single-stage transitions) *At a single stage with $p \leq q$:*

Case 1: $a_A \leq a_B$. (1a) No overlap *($a_A + p \leq a_B$): both priorities give $C(A) = a_A + p$, $C(B) = a_B + q$.* (1b) Overlap, A-first *($a_A + p > a_B$): $C(A) = a_A + p$, $C(B) = a_A + p + q$.* (1c) Overlap, B-first *($a_A + p > a_B$): $C(B) = a_B + q$, $C(A) = a_A + p + q$.*

Case 2: $a_B < a_A$. (2a) No overlap *($a_B + q \leq a_A$): both priorities give $C(B) = a_B + q$, $C(A) = a_A + p$.* (2b) Overlap, A-first *($a_B + q > a_A$): $C(A) = a_A + p$, $C(B) = a_B + p + q$.* (2c) Overlap, B-first *($a_B + q > a_A$): $C(B) = a_B + q$, $C(A) = a_B + p + q$.*

Lemma 15 (Single-stage flowtime optimality) *At a single stage with $p \leq q$ and $a_A \leq a_B$:* (i) *A-first minimizes $C(A) + C(B)$;* (ii) *under A-first, $C(A) \leq C(B)$.*

Proof No overlap: both priorities coincide and $C(A) = a_A + p \le a_B + q = C(B)$. Overlap: the A-first sum is $2a_A + 2p + q$ and the B-first sum is $a_A + a_B + p + 2q$; the difference $(a_A - a_B) + (p - q) \le 0$. Under A-first, $C(A) = a_A + p < a_A + p + q = C(B)$. □

7.2 The sync value function

Under σ^* with initial arrivals (α, β), $\alpha \le \beta$, job A is never preempted and completes each stage first (Lemma 15(ii) inductively), giving $C^*_\ell(A) = \alpha + \sum_{s \le \ell} p_s$ and $C^*_\ell(B) = \max(C^*_\ell(A), C^*_{\ell-1}(B)) + q_\ell$ with $C^*_0(B) = \beta$.

Definition 16 (Sync value) For an L-stage concordant chain and arrivals (α, β) with $\alpha \le \beta$, let $\Phi_L(\alpha, \beta) = C^*_L(A) + C^*_L(B)$, so that

$$\Phi_L(\alpha, \beta) = \Phi_{L-1}\big(\alpha + p_1,\ \max(\alpha + p_1, \beta) + q_1\big), \qquad \Phi_0(\alpha, \beta) = \alpha + \beta,$$

where Φ_{L-1} refers to the chain with stage 1 removed.

Lemma 17 (Monotonicity) *$\Phi_L(\alpha, \beta)$ is non-decreasing in both arguments (for $\alpha \le \beta$).*

Proof Induction on L. For $L = 0$, $\Phi_0 = \alpha + \beta$. For $L \ge 1$, $\Phi_L = \Phi_{L-1}(f, g)$ with $f = \alpha + p_1$ and $g = \max(\alpha + p_1, \beta) + q_1$ both non-decreasing in (α, β), and $f \le g$; conclude by the inductive hypothesis. □

Lemma 18 (Sum-preserving monotonicity) *For fixed $T = \alpha + \beta$ with $\alpha \le \beta$, $\Phi_L(\alpha, T - \alpha)$ is non-decreasing in α.*

Proof Induction on L. For $L = 0$ the value is the constant T. For $L \ge 1$, $\Phi_L(\alpha, T - \alpha) = \Phi_{L-1}(f(\alpha), g(\alpha))$ with $f(\alpha) = \alpha + p_1$ increasing and $g(\alpha) = \max(\alpha + p_1, T - \alpha) + q_1$ piecewise linear: g decreases while $2\alpha + p_1 \le T$ and increases thereafter. On the decreasing regime, $f + g = T + p_1 + q_1$ is constant with f increasing, so the inductive hypothesis (applied at sum $T + p_1 + q_1$) gives the claim; on the increasing regime both arguments are non-decreasing and Lemma 17 applies. □

7.3 Two interlocking inductions

Proposition 19 (A-first arrivals) *For any L-stage concordant chain, arrivals (α, β) with $\alpha \le \beta$, and any schedule σ: $\Phi_L(\alpha, \beta) \le C^\sigma_L(A) + C^\sigma_L(B)$.*

Lemma 20 (B-first arrivals) *For any L-stage concordant chain, arrivals (a, b) with $a > b \ge 0$ (A arrives at a, B at b), and any schedule σ: $C^\sigma_L(A) + C^\sigma_L(B) \ge \Phi_L(b, a)$.*

Theorem 13 is Proposition 19 with $\alpha = \beta = 0$. Lemma 20 says the "wrong" arrival order cannot beat the sync value computed from the sorted arrivals. We prove both simultaneously by induction on L.

Proof of Proposition 19 and Lemma 20 **Base case** $L = 0$**.** Both sides equal $\alpha + \beta$ (resp. $a + b$).

Base case $L = 1$**.** For Proposition 19, Lemma 15(i) states exactly that A-first minimizes the single-stage sum. For Lemma 20 ($a > b$), check the cases of Lemma 14: *(2a)* the sum is $a+b+p_1+q_1$; since $b+p_1 \le b+q_1 \le a$, $\Phi_1(b,a) = (b+p_1)+(a+q_1)$, with equality. *(2b)* the sum is $a+b+2p_1+q_1$. If $a \ge b+p_1$ then $\Phi_1(b,a) = a+b+p_1+q_1 \le$ the sum; if $a < b+p_1$ then $\Phi_1(b,a) = 2b+2p_1+q_1 \le$ the sum since $b < a$. *(2c)* the sum is $2b+p_1+2q_1$. If $a \ge b+p_1$ then $\Phi_1(b,a) = a+b+p_1+q_1 \le 2b+p_1+2q_1$ since $a-b \le q_1$ (overlap); if $a < b+p_1$ then $\Phi_1(b,a) = 2b+2p_1+q_1 \le 2b+p_1+2q_1$ since $p_1 \le q_1$.

Inductive step for Proposition 19. Arrivals (α,β), $\alpha \le \beta$; stage-1 priority π_1. Recall $\Phi_L(\alpha,\beta) = \Phi_{L-1}(\alpha+p_1, \max(\alpha+p_1,\beta)+q_1)$.

Case $\pi_1 = A$. Stage-1 outputs are $C_1(A) = \alpha+p_1 \le C_1(B) = \max(\alpha+p_1,\beta)+q_1$ (cases 1a/1b), and the $(L-1)$-stage Proposition 19 gives $C_L^\sigma \ge \Phi_{L-1}(C_1(A), C_1(B)) = \Phi_L(\alpha,\beta)$.

Case $\pi_1 = B$. Without overlap ($\alpha+p_1 \le \beta$) the outputs coincide with the A-first outputs and the previous case applies. With overlap, outputs are $C_1(A) = \alpha+p_1+q_1 > C_1(B) = \beta+q_1$ (case 1c), so the $(L-1)$-stage Lemma 20 gives $C_L^\sigma \ge \Phi_{L-1}(\beta+q_1,\, \alpha+p_1+q_1)$. Since overlap means $\alpha+p_1 > \beta$, $\Phi_L(\alpha,\beta) = \Phi_{L-1}(\alpha+p_1,\, \alpha+p_1+q_1)$; the second arguments agree, and $\alpha+p_1 \le \beta+q_1$ (from $\alpha \le \beta$, $p_1 \le q_1$), so Lemma 17 in the first argument completes the case.

Inductive step for Lemma 20. Arrivals (a,b), $a > b$; stage-1 priority π_1. Write $\mu = \max(b+p_1, a)$, so $\Phi_L(b,a) = \Phi_{L-1}(b+p_1, \mu+q_1)$.

Case 2a (no overlap, $b+q_1 \le a$*).* Both priorities give $C_1(A) = a+p_1 > C_1(B) = b+q_1$, and the $(L-1)$-stage Lemma 20 gives $C_L^\sigma \ge \Phi_{L-1}(b+q_1, a+p_1)$. Here $\mu = a$ (as $b+p_1 \le b+q_1 \le a$), so $\Phi_L(b,a) = \Phi_{L-1}(b+p_1, a+q_1)$. The pairs $(b+q_1, a+p_1)$ and $(b+p_1, a+q_1)$ share the sum $a+b+p_1+q_1$, with $b+p_1 \le b+q_1 \le a+p_1$, so Lemma 18 gives $\Phi_{L-1}(b+q_1, a+p_1) \ge \Phi_{L-1}(b+p_1, a+q_1)$.

Case 2b (overlap, $\pi_1 = A$*).* Outputs $C_1(A) = a+p_1 < C_1(B) = b+p_1+q_1$ (using $b+q_1 > a$), so the $(L-1)$-stage Proposition 19 gives $C_L^\sigma \ge \Phi_{L-1}(a+p_1,\, b+p_1+q_1)$. If $a \ge b+p_1$ ($\mu = a$): the pairs $(a+p_1, b+p_1+q_1)$ and $(b+p_1, a+p_1+q_1)$ share the sum $a+b+2p_1+q_1$, each has first argument $\le$ second (for the first pair because $a < b+q_1$), and $a+p_1 \ge b+p_1$; Lemma 18 then Lemma 17 (second argument, $a+p_1+q_1 \ge a+q_1$) give

$$\Phi_{L-1}(a{+}p_1, b{+}p_1{+}q_1) \ge \Phi_{L-1}(b{+}p_1, a{+}p_1{+}q_1) \ge \Phi_{L-1}(b{+}p_1, a{+}q_1) = \Phi_L(b,a).$$

If $a < b+p_1$ ($\mu = b+p_1$): by Lemma 17 in the first argument ($a+p_1 > b+p_1$), $\Phi_{L-1}(a+p_1, b+p_1+q_1) \ge \Phi_{L-1}(b+p_1, b+p_1+q_1) = \Phi_L(b,a)$.

Case 2c (overlap, $\pi_1 = B$*).* Outputs $C_1(B) = b+q_1 < C_1(A) = b+p_1+q_1$, so the $(L-1)$-stage Lemma 20 gives $C_L^\sigma \ge \Phi_{L-1}(b+q_1,\, b+p_1+q_1)$. If $a < b+p_1$ ($\mu = b+p_1$): monotonicity in the first argument ($b+q_1 \ge b+p_1$) gives $\Phi_{L-1}(b+q_1, b+p_1+q_1) \ge \Phi_{L-1}(b+p_1, b+p_1+q_1) = \Phi_L(b,a)$. If $a \ge b+p_1$ ($\mu = a$), split on $b+q_1 \lesseqgtr a+p_1$. If $b+q_1 \le a+p_1$: monotonicity in the second argument ($b+p_1+q_1 \ge a+p_1$, from $b+q_1 > a$) gives $\Phi_{L-1}(b+q_1, b+p_1+q_1) \ge \Phi_{L-1}(b+q_1, a+p_1)$, and the pairs $(b+q_1, a+p_1)$, $(b+p_1, a+q_1)$ share their sum with $b+p_1 \le b+q_1 \le a+p_1$, so Lemma 18 finishes. If $b+q_1 > a+p_1$: by Lemma 17 (componentwise, since $b+q_1 \ge a+p_1$ and $b+p_1+q_1 \ge b+q_1$), $\Phi_{L-1}(b+q_1, b+p_1+q_1) \ge \Phi_{L-1}(a+p_1, b+q_1)$; the pairs $(a+p_1, b+q_1)$ and $(b+p_1, a+q_1)$ share their sum with $a+p_1 \ge b+p_1$ and first $\le$ second in each, so Lemma 18 gives $\Phi_{L-1}(a+p_1, b+q_1) \ge \Phi_{L-1}(b+p_1, a+q_1) = \Phi_L(b,a)$.

This closes both inductions and proves Theorem 13. $\square$

Remark 21 (Sharpness) Concordance is essential, already for unweighted flowtime: for the discordant works $w = [(1,6),(6,1),(6,1)]$ with $k = 2$, the best synchronized total flowtime is 13.5 while the per-stage orders $(1,2) \mid (2,1) \mid (2,1)$ attain 11.5 (and Proposition 6's instance fails for weighted flowtime). Two jobs are essential as well: for $L = 2$, $n = 3$, $k = 2$,

$w = [(4,5,6),\ (1,9,10)]$ (strictly concordant), the synchronized lighter-first schedule attains total flowtime 25.5, while the per-stage orders $(2,1,3) \mid (1,2,3)$ attain 25.0 by letting the lightest job preempt a medium job at stage 2. In an exhaustive sweep this failure is rare: it is the sole counterexample among 14,400 strictly concordant $n = 3$, $L = 2$ configurations with work values 1–10, and 175,616 strictly concordant $n = 3$, $L = 3$ configurations with work values 1–8 contain no flowtime failure. The $n = 2$ theorem is itself verified exhaustively for $L \in \{2, 3\}$, $k = 2$, work values 1–8: all 784 strictly concordant two-stage and 21,952 strictly concordant three-stage configurations (46,656 at $L = 3$ when ties are allowed) show zero flowtime failures.

8 Practice: policy benchmarks

The negative results say that at depth three or more, full synchronization is no longer free. They do not say what a practitioner should do. This section evaluates the policy we recommend and quantifies both its aggregate cost and the structure of the residual gap. All statistics below are produced by exhaustive or seeded-random benchmarks in the companion artifact (see Code availability); the model is that of Definition 1.

Definition 22 (Composite policy: source-sync with FIFO downstream) Fix one order π. Every source stage uses priority π. Every non-source stage serves first-in-first-out by arrival time, breaking ties by π.

The policy asks for global agreement only where work originates; downstream teams need no ranking at all—they simply take work in the order it reaches them. On single-source topologies this is provably not a relaxation of full synchronization but a reimplementation of it:

Lemma 23 (Propagation of source order) *Let G have a single source. Then for every work assignment, the composite policy with order π produces exactly the completion times of the fully synchronized schedule $\beta_v \equiv \pi$, at every stage.*

Proof By induction over a topological order, every stage's arrival vector is weakly π-ordered and its effective service order is π. At the source, all jobs arrive at time 0 and the policy uses π by definition; by Lemma 2(v), completions are π-ordered. At a non-source stage v, each predecessor's completions are π-ordered by induction, so the componentwise maximum defining a_v is weakly π-ordered; FIFO-by-arrival with ties broken by π therefore sorts jobs exactly in the order π. With π-ordered arrivals and priority π, no job ever overtakes another in service (Lemma 2(v)): completions follow the max-plus recurrence and are again π-ordered—and they coincide with the synchronized schedule's completions at v, since that schedule has the same arrivals (inductively) and the same priority order. □

Remark 24 The argument extends verbatim to multi-source DAGs when all sources share the order π and all jobs are released simultaneously, since componentwise maxima of π-ordered vectors are π-ordered. The benchmarks below corroborate this exactly: per order π, the composite policy and full synchronization produced identical sink makespans on all 561,633 chain and fork-join–fork-join instances tested (per-π mismatch count zero)—the chain and fork-join–fork-join families of Table 2 plus a 4,096-instance exhaustive fork-join–fork-join sweep with works 1–2—including, in particular, the two-source fork-join–fork-join topology. Downstream discipline is a non-issue once the sources are aligned.

Aggregate quality.

On the exhaustive diamond family of Section 6 ($n = 2$, $k_v = 2$, all works in $\{1, \ldots, 4\}^8$; 65,536 instances), with the source order chosen by enumeration and compared against the enumerated optimum over all per-stage orders, the composite policy is optimal for the makespan in 99.45% of instances, with mean gap 0.058% and maximum gap 14.29%; for total flowtime at the sink it is optimal in 99.41% of instances, with mean gap 0.036% and maximum gap 11.76%.

Across topology families.

The same pattern holds beyond the diamond. Table 2 reports the identical benchmark on exhaustive and seeded-random families over three-stage chains and the two-source fork-join–fork-join topology of Section 6 ($k_v = 2$ throughout). In every family the policy's optimality rate and full synchronization's failure rate sum to one: by the per-π identity of Remark 24 the composite policy is optimal exactly where full synchronization is, so it inherits full synchronization's failures and nothing else. Failure rates grow with the work range and with n—from 0.55% on the exhaustive diamond family to 10.05% on random three-job fork-join–fork-join instances—but the conditional cost where synchronization fails stays in one band: mean gap 5.5–10.4%, maximum 12.5–20%.

Table 2 Composite policy against the enumerated optimum across topology families (chains are three-stage; ranges are integer work values; exh. = exhaustive, rnd. = seeded random). "Pol. opt." is the fraction of instances where the best-π policy makespan equals the optimum; "sync fails" is the fraction where the best fully synchronized makespan is suboptimal (the two columns sum to 100%); the gap columns describe the relative excess over the optimum on the failing instances.

Family	Inst.	Pol. opt.	Sync fails	Gap mean	Gap max
Diamond n=2, 1–4 (exh.)	65,536	99.45%	0.55%	10.5%	14.3%
Chain n=2, 1–4 (exh.)	4,096	99.07%	0.93%	10.4%	14.3%
Chain n=3, 1–6 (rnd.)	20,000	94.03%	5.97%	6.3%	20.0%
FJFJ n=2, 1–3 (exh.)	531,441	98.68%	1.32%	8.9%	12.5%
FJFJ n=3, 1–6 (rnd.)	2,000	89.95%	10.05%	5.5%	15.0%

Cost of synchronization where it fails.

On the same family, full synchronization is makespan-suboptimal on 362 instances (0.55%); on those instances the gap to the optimum has mean 10.5%, median 10.0%, and maximum 14.3%. This converts the qualitative "dominance can fail" of Theorem 9 into an expected cost: failures are rare but material when present. The maximum is attained by the embedded chain instance of Table 1 (synchronized 4 against optimal 3.5: $1/7 \approx 14.3\%$ above the optimum, i.e. the optimum is $1/8$ below the synchronized makespan, matching the asymptotic constant of Theorem 9). By Lemma 23, the composite policy coincides with full synchronization on this single-source family, so its worst instances are the same 0.55%: capturing the residual 10–14% there requires deliberate priority overtaking—expediting the work-inverted job—which no arrival-order discipline reproduces.

Robustness to estimation error.

Rankings are made on estimates. On diamonds with $n = 3$ (5,000 seeded instances; estimated works drawn from $\{10, 20, \dots, 60\}$; true works equal to the estimate scaled by an independent uniform factor on $[0.5, 1.5]$, i.e. up to $\pm 50\%$ error), choosing π from the *estimates*, executing on the *true* works with FIFO adapting to actual arrivals at runtime, and measuring against the clairvoyant optimum on true works: the composite policy is optimal outright in 50.1% of instances, with mean gap 3.03% and maximum 32.5%. The picture is stable across topologies: under the same protocol the mean gap is 3.17% on three-stage chains (5,000 instances; optimal in 50.9%, maximum 33.6%) and 4.11% on the two-source fork-join–fork-join family (1,000 instances; optimal in 38.8%, maximum 29.8%); in all three families the executed policy and the executed fully synchronized schedule again had identical makespans on every instance. The bulk of alignment's benefit survives severe estimation error; the residual 3–4% is the price of uncertainty itself, not of the policy.

Managerial reading.

Agree on one ranked list where work enters the system; let every downstream team serve work in the order it arrives. On single-source delivery structures this reproduces the fully synchronized schedule automatically—no further coordination meetings, no per-team re-ranking—and work reaches the final stage as soon as it would under organization-wide synchronization. The residual upside over that baseline is rare (0.55–10% of instances across the families of Table 2), worth roughly 5–10% when present, and lives in identifiable cross-stage work inversions: an item that is light for the upstream team but heavy downstream (Remark 7). The playbook is therefore: align the source; let arrival order govern everything below it; and treat inversions as named exceptions to expedite deliberately, rather than re-sequencing every team's queue.

9 Discussion

Table 3 assembles the corrected dominance landscape. The single organizing fact is Corollary 11: for the makespan, the boundary of safe synchronization is depth, not width. The diamond and fork-join rows of our earlier preprint's landscape table claimed computational support for dominance on those topologies; both rows are corrected here, and the depth criterion subsumes the would-be taxonomy of tree and layered-DAG special cases (any such topology is safe precisely when it contains no directed three-stage path).

Table 3 Dominance landscape (corrected). All entries proved in this paper or in Bowman (2026).

Model	Property	Status
Two-stage assembly (Bowman, 2026)	Pointwise (downstream inherited)	Holds
Depth-2 DAG, any sink (Thm. 10)	Makespan dominance	Holds
Chain, $L = 2$ (Thm. 5)	Makespan dominance	Holds
Chain, $L = 2$ (Rem. 8)	Pointwise (fully synchronized)	Fails
Chain, $L \geq 3$ (Prop. 6)	Makespan / weighted flowtime	Fails
DAG with a 3-path (Thm. 9)	Makespan dominance	Fails
Concordant chain, $n = 2$ (Thm. 13)	Total flowtime	Holds
Concordant chain, $n \geq 3$ (Rem. 21)	Total flowtime	Fails

One adjacent question is settled computationally: the depth-three failures *survive* the integer, non-preemptive divisible-work model of Bowman (2026), but only barely. Chaining that model through three stages ($k = 2$ workers per stage; each job's integer stage work split freely into integer per-worker portions; no preemption), an exhaustive sweep of all 4,096 two-job instances with stage works at most 4 finds exactly two makespan-dominance failures—a single instance up to job relabeling, $w = [(1, 3), (4, 1), (1, 3)]$, with synchronized best 6 against optimum 5; the optimum requires both an order inversion at the final stage and an uneven integer split of the heavy middle-stage work. The failure is a granularity knife-edge: scaling that instance's works by 2, 3, or 4 closes the gap entirely (synchronized = optimum = $9, 15, 18$), and the fluid counterexample of Proposition 6 never fails in the integer model at any tested scale (work multipliers up to 8), even though its fluid gap grows linearly. Indivisibility removes the mid-stage overtaking the fluid counterexamples exploit, so the depth boundary is real but attenuated by integrality: it bites only at work granularities comparable to the worker count.

Two questions remain open. First, whether deeper embedded chains can push the asymptotic optimality gap beyond 1/8 of the synchronized makespan. Second, stochastic processing times: the robustness benchmark of Section 8 suggests the source-alignment policy degrades gracefully under estimation error, but a distributional theory of synchronization dominance is untouched.

Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests. The author declares no competing interests.

Author contributions. The sole author conceived the work, developed the proofs, and wrote the manuscript.

Data availability. The verification and benchmark data supporting the quantitative claims of Sections 6–8 are available in the repository below.

Code availability. The simulation and benchmark code is available at https://github.com/ebowman/synchronization-paper-artifact and archived at https://doi.org/10.5281/zenodo.20075388.

Generative AI disclosure. During the preparation of this work, the author used Claude for critique of exposition, structural feedback, and language revision. The author reviewed and edited the manuscript and takes full responsibility for the content.

References

Bowman, E.: Permutation-schedule dominance for two-stage assembly flow shops with divisible work. Submitted to Journal of Scheduling; preprint doi:10.5281/zenodo.20075388 (2026)

Lee, C.-Y., Cheng, T.C.E., Lin, B.M.T.: Minimizing the makespan in the 3-machine assembly-type flowshop scheduling problem. Management Science **39**(5), 616–625 (1993) https://doi.org/10.1287/mnsc.39.5.616

Potts, C.N., Sevast'janov, S.V., Strusevich, V.A., Van Wassenhove, L.N., Zwaneveld, C.M.: The two-stage assembly scheduling problem: Complexity and approximation. Operations Research **43**(2), 346–355 (1995) https://doi.org/10.1287/opre.43.2.346

Tozkapan, A., Kırça, Ö., Chung, C.-S.: A branch and bound algorithm to minimize the total weighted flowtime for the two-stage assembly scheduling problem. Computers & Operations Research **30**(2), 309–320 (2003) https://doi.org/10.1016/S0305-0548(01)00098-3

Hariri, A.M.A., Potts, C.N.: A branch and bound algorithm for the two-stage assembly scheduling problem. European Journal of Operational Research **103**(3), 547–556 (1997) https://doi.org/10.1016/S0377-2217(96)00312-8

Hadda, H., Dridi, N., Hajri-Gabouj, S.: On the two-stage assembly flow shop problem. TOP **32**(2), 224–244 (2024) https://doi.org/10.1007/s11750-024-00664-0

Koulamas, C., Kyparisis, G.J.: A note on the two-stage assembly flow shop scheduling problem with uniform parallel machines. European Journal of Operational Research **182**(2), 945–951 (2007) https://doi.org/10.1016/j.ejor.2006.07.027

Drozdowski, M.: Scheduling for Parallel Processing. Springer, London (2009). https://doi.org/10.1007/978-1-84882-310-5

Hatami, S., Ruiz, R., Andrés-Romano, C.: The distributed assembly permutation flowshop scheduling problem. International Journal of Production Research **51**(17), 5292–5308 (2013) https://doi.org/10.1080/00207543.2013.807955

Zhang, Z., Chen, R.: Multi-objective complex product assembly scheduling problem considering parallel team and worker skills. Journal of Manufacturing Systems **62**, 876–893 (2022) https://doi.org/10.1016/j.jmsy.2022.05.003

Komaki, G.M., Sheikh, S., Malakooti, B.: Flow shop scheduling problems with assembly operations: A review and new trends. International Journal of Production Research **57**(10), 2926–2955 (2019) https://doi.org/10.1080/00207543.2018.1550269

Al-Anzi, F.S., Allahverdi, A.: A self-adaptive differential evolution heuristic for two-stage assembly scheduling problem to minimize maximum lateness with setup times. European Journal of Operational Research **182**(1), 80–94 (2007) https://doi.org/10.1016/j.ejor.2006.09.011

Graham, R.L., Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G.: Optimization and approximation in deterministic sequencing and scheduling: A survey. Annals of Discrete Mathematics **5**, 287–326 (1979) https://doi.org/10.1016/S0167-5060(08)70356-X